Jen v4 使い方ガイド

完了は、証拠で決める。

jen-marketplace v4.0.0

Table of Contents

1. v3 から何が変わったか

変わったのは「完了」の意味だけです。 モード（route / conduct / repair / review / release / longrun）も、18 体のエージェントも、委譲の流れもそのままです。

	v3 まで	**v4**
完了の判定	Verifier が ACCEPT と言ったら完了	宣言した主張すべてに証拠が付いたら完了
判定するのは	LLM	判定はモデル、合成は **Python**
「テストが通った」の根拠	エージェントの報告	台帳が自分で実行した終了コード
完了を止めるのは	プロンプト（お願い）	**Stop hook**（**exit 2** で差し戻す）
受入条件の置き場	`.jen/mission.md`（文章）	**`.jen/claims.json`**（状態を持つ台帳）

v3 の「Verifier が ACCEPT するまで完了と言わない」は、**LLM** がそう振る舞ってくれることを期待していただけでした。v4 はそこだけを機械に移します。

> 何も宣言しなければ、**v4** は **v3** と同じように動きます。 台帳が無いセッションはゲートを通りません（ただし「ゲート無しで終わった」と記録されます）。

2. 最小の流れ

`$H` は hooks のパスです。プラグインとして入れている場合は `$CLAUDE_PLUGIN_ROOT/hooks` になります。

```
H="$CLAUDE_PLUGIN_ROOT/hooks"
```

① 受入条件を主張に分解して宣言する

「サインアップ画面を改善する」のような文章ではなく、**1** つずつ独立に検証できる形に割ります。

```
python3 "$H/evidence_ledger.py" init --tier sonnet \
  --claim "C1=lint / typecheck / test が通る" \
  --claim "C2=初回画面に次アクションが表示される"
```

--tier はセンサーの頻度を決めます（第 5 節）。

② 作業する

ここは v3 と同じです。Jen が分解し、委譲し、実装します。

③ 証拠を付ける

「できました」と報告しても主張は閉じません。 台帳に実行させます。

```
python3 "$H/evidence_ledger.py" run --claim C1 --cmd "npm test"
```

台帳が npm test を自分で実行し、終了コードを捕まえます。出力は .jen/logs/evidence/ に残ります。

④ 状態を見る

```
python3 "$H/evidence_ledger.py" status
```

全主張が MATCH なら exit 0、そうでなければ exit 1 です。

3. 証拠には強度がある

文字列を出力するだけのゲートは破られます。 実際に、Stop Hook が出力中の proof 文字列を見ていたところ、モデルはその文字列を出力するだけでバイパスした、という報告があります（コキエ 2026-03）。

そこで v4 は証拠を 3 段階に分けます。

強度	作り方	誰が作れるか
weak	attest --kind note（申告）	エージェント
medium	attest --kind artifact（ファイル。台帳が sha256 を計算）	エージェント
strong	**run --cmd**（台帳が自分で実行）	台帳だけ

既定では require=medium なので、申告（**weak**）だけでは主張は閉じません。

```
# これは PARTIAL どまり（weak）
python3 "$H/evidence_ledger.py" attest --claim C2 --kind note --detail "実装した"

# これは MATCH（medium）
python3 "$H/evidence_ledger.py" attest --claim C2 --kind artifact --ref screenshot.png
```

重要な主張には strong を要求できます。

```
python3 "$H/evidence_ledger.py" claim-add --id C3 \
  --text "既存テストを壊していない" --require strong
```

強い証拠を宣言する経路はありません。 run だけが作れます。

4. コマンド一覧

コマンド	何をするか
`init`	台帳を作る。`--tier --require --max-delegations --force`
`claim-add`	主張を足す。`--id --text --require`
`attest`	弱い／中程度の証拠を付ける。`--kind note|artifact`
`run`	台帳が自分で実行し、終了コードを強い証拠にする。`--cmd --expect-exit`
`status`	被覆状況を表示（全 MATCH で exit 0）
`gate`	完了可否の判定（Stop hook が使う。塞ぐときは exit 2）
`budget`	委譲回数を hook の実ログから数える
`revise`	問いが変わったとき版を切る。被覆はリセットされる
`tier`	階層を変えてセンサーの頻度を切り替える

5. センサーをいつ走らせるか

独立した検収は常に必要ですが、頻度はモデルで変えます。

tier	sensor_mode	使い方
`haiku / sonnet`	`continuous`	工程ごとに `run` で被覆を進める
`opus / fable`	`final_gate`	工程中は回さず、最後に 1 回まとめて検める

根拠は独立した 2 つの実験です。フロンティアモデルはハーネス有無どちらでも成功する一方、低コストモデルは素で 3/10 がハーネス付きで 10/10 になった（Jain 2026-10）。Anthropic も Opus 4.6 の改善を理由に evaluator を最終 1 回に移しています（2026-03）。

安いモデルほどハーネスが元を取ります。

6. ゲートに差し戻されたら

完了しようとして `exit 2` で止められたときは、次の 3 つのどれかです。

1. 証拠が足りない → 未被覆の主張に `run` で強い証拠を付ける
2. 主張そのものが間違っていた → 人間に確認してから `revise` で版を切る（被覆はリセットされます）
3. 解けない → Human Gate として「何ができて・なぜ止まり・人間は何をすればいいか」を報告する

```
python3 "$H/evidence_ledger.py" revise --note "受入条件に対応端末を追加した"
```

7. ゲートを切る（人間の判断）

ゲートは人間が明示的に切れます。.jen/claims.json に "gate": "off" を入れてください。切ったことは記録に残ります（stop-events.jsonl の reason: gate_off_by_human）。

エージェント側から切る経路は用意していません。

8. 資源を数える（Lean 版）

jen-classic（Lean 版）は「同じ成果をより少ない資源で」を目的にしています。委譲回数を数えられます。

```
python3 "$H/evidence_ledger.py" init --tier haiku --max-delegations 8 --claim "C1=..."
python3 "$H/evidence_ledger.py" budget
```

サブエージェントは呼び出しごとにまっさらなコンテキストで始まるので、委譲回数が支配的なコストです。超過しても強制停止はしません（点検シグナルです）。

測れないときは「判定不能」と表示します。黙って **0** とは言いません。

9. やってみる（この通りに動きます）

以下は実際に動く一連の流れです。空のディレクトリで試してください。

```
# VERIFY-BLOCK
set -e
H="${H:?hooks のパスを H に設定してください}"

# 1. 主張を 2 つ宣言する
python3 "$H/evidence_ledger.py" init --force --tier sonnet \
  --claim "C1=ビルドが通る" --claim "C2=成果物が存在する"

# 2. 申告（weak）は PARTIAL どまり
python3 "$H/evidence_ledger.py" attest --claim C1 --kind note --detail "実装した"
python3 "$H/evidence_ledger.py" status || echo "→ まだ閉じていない（想定どおり）"

# 3. ゲートは塞ぐ
python3 "$H/evidence_ledger.py" gate 2>/dev/null && echo "通ってはいけない" && exit 1
echo "→ ゲートが塞いだ（想定どおり）"

# 4. 台帳に実行させる＝強い証拠
python3 "$H/evidence_ledger.py" run --claim C1 --cmd "true"

# 5. 成果物を作って中程度の証拠を付ける
echo "artifact" > out.txt
python3 "$H/evidence_ledger.py" attest --claim C2 --kind artifact --ref out.txt

# 6. 全主張が揃ったのでゲートが通る
python3 "$H/evidence_ledger.py" status
python3 "$H/evidence_ledger.py" gate
```

```
echo "→ 完了が許された"

# 7. 問いが変われば被覆はリセットされる
python3 "$H/evidence_ledger.py" revise --note "受入条件が変わった"
python3 "$H/evidence_ledger.py" gate 2>/dev/null && echo "リセットされていない" && exit
1
echo "→ 版を切ったので被覆が消えた（想定どおり）"

# 8. 資源を数える
python3 "$H/evidence_ledger.py" budget
```

このブロックは **CI** 相当の検証で実際に実行しています。 書いてあるとおりに動かなければ、このガイドは出せません。
